Farnell AN AVNET COMPANY Newark AN AVNET COMPANY element14

EN - For pricing and availability in your local country please visit one of the below links:

DE - Informationen zu Preisen und Verfügbarkeit in Ihrem Land erhalten Sie über die unten aufgeführten Links:

FR - Pour connaître les tarifs et la disponibilité dans votre pays, cliquez sur l'un des liens suivants:

107990193

EN
This Datasheet is presented by the manufacturer

DE
Dieses Datenblatt wird vom Hersteller bereitgestellt

FR
Cette fiche technique est présentée par le fabricant

ReSpeaker USB Mic Array

An out-of-the-box voice pick-up device is the voice of the customer.

During the past year, Respeaker Mic Array V2.0 [https://www.seeedstudio.com/ReSpeaker-Mic-Array-v2-0.html] has been sold out for more than 10K units in the format of the development board. Customers keep requesting a complete device

with an enclosure, which is challenging for them to design it, considering the acoustic principles.

And here Seeed provides the answer with ReSpeaker USB Mic Array:

- An out-of-box device with a well-designed acoustic structure brings the flexibility for the customer to build in their solution.
- Mold injected enclosure available, saves the time to go to the market and the mold cost.

The difference between the PCBA inside ReSpeaker USB Mic Array and Respeaker Mic Array V2.0:

- Optimized power circuit
- Move the audio jack and micro USB port to the backside.

Get One Now

[https://www.seeedstudio.com/ReSpeaker-USB-Mic-Array-p-4247.html]

Features

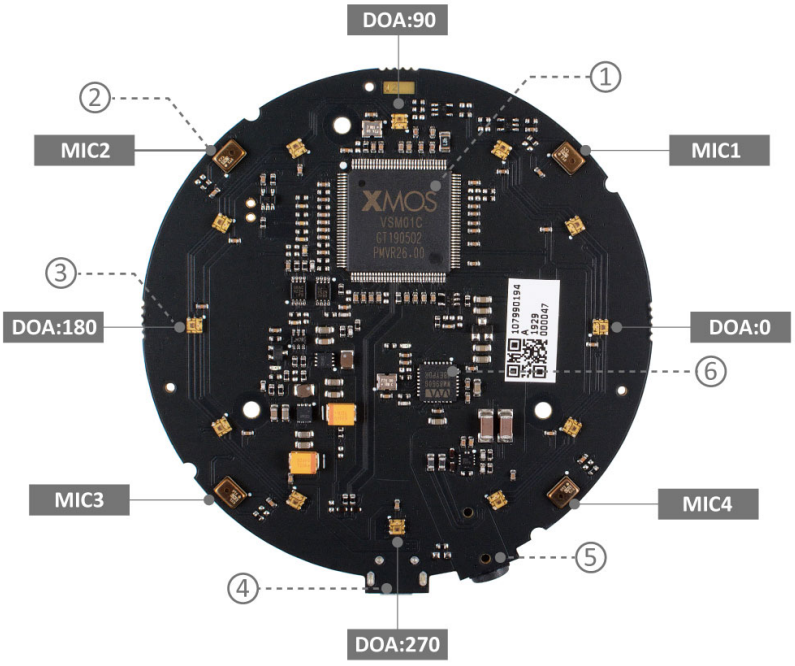
- Far-field voice capture
- Support USB Audio Class 1.0 (UAC 1.0)
- Four microphones array
- 12 programmable RGB LED indicators
- Speech algorithms and features
    - Voice Activity Detection

- Direction of Arrival
- Beamforming
- Noise Suppression
- De-reverberation
- Acoustic Echo Cancellation

Specification

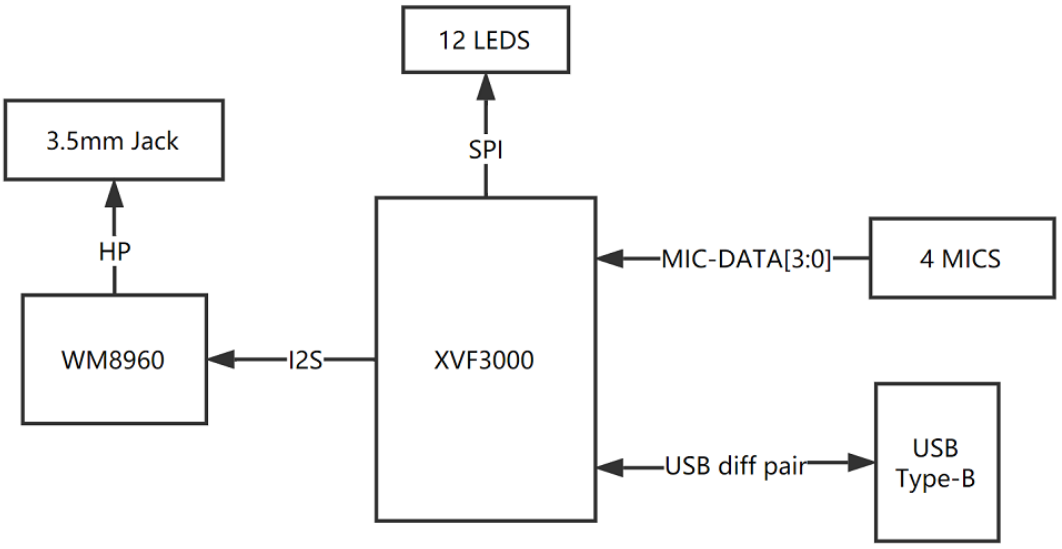
- XVF-3000 from XMOS
- 4 high performance digital microphones
- Supports Far-field Voice Capture
- Speech algorithm on-chip
- 12 programmable RGB LED indicators
- Microphones: ST MP34DT01TR-M
- Sensitivity: -26 dBFS (Omnidirectional)
- Acoustic overload point: 120 dBSPL
- SNR: 61 dB
- Power Supply: 5V DC from Micro USB
- Dimensions: 70mm (Diameter)
- 3.5mm Audio jack output socket
- Power consumption: 5V, 180mA with led on and 170mA with led off
- Max Sample Rate: 16Khz

Hardware Overview



- ① **XMOS XVF-3000:** It integrates advanced DSP algorithms that include Acoustic Echo Cancellation (AEC), beamforming, dereverberation, noise suppression and gain control.
- ② **Digital Microphone:** The MP34DT01-M is an ultra-compact, lowpower, omnidirectional, digital MEMS microphone built with a capacitive sensing element and an IC interface.
- ③ **RGB LED:** Three-color RGB LED.
- ④ **USB Port:** Provide the power and control the mic array.
- ⑤ **3.5mm Headphone jack:** Output audio, We can plug active speakers or Headphones into this port.
- ⑥ **WM8960:** The WM8960 is a low power stereo codec featuring Class D speaker drivers to provide 1 W per channel into 8 W loads.

System Diagram



Applications

- USB Voice Capture
- Smart Speaker
- Intelligent Voice Assistant Systems
- Voice Recorders
- Voice Conferencing System
- Meeting Communicating Equipment
- Voice Interacting Robot
- Car Voice Assistant
- Other Voice Interface Scenarios

Getting Started

> **Note**
> ReSpeaker USB Mic Array is compatiable with Windows, Mac, Linux systems andriod. The below scripts are tested on Python2.7.

Update Firmware

Here is the table for the differences.

Firmware	Channels	Note
1_channel_firmware.bin	1	processed audio for ASR
6_channels_firmware.bin	6	channel 0: processed audio for ASR, channel 1-4: 4 microphones' raw data, channel 5: playback（factory firmware）

For Linux: The Mic array supports the USB DFU. We develop a python script dfu.py to update the firmware through USB.

```
sudo apt-get update
sudo pip install pyusb click
git clone https://github.com/respeaker/usb_4_mic_array.gi
cd usb_4_mic_array
sudo python dfu.py --download 6_channels_firmware.bin  #

# if you want to use 1 channel,then the command should be

sudo python dfu.py --download 1_channel_firmware.bin
```

Here is the firmware downloading result.

```
pi@raspberrypi:~/usb_4_mic_array $ sudo python dfu.py --download default_firmware.bin
entering dfu mode
found dfu device
downloading
150336 bytes
done
```

For Windows/Mac: We do not suggest use Windows/Mac and Linux vitual machine to update the firmware.

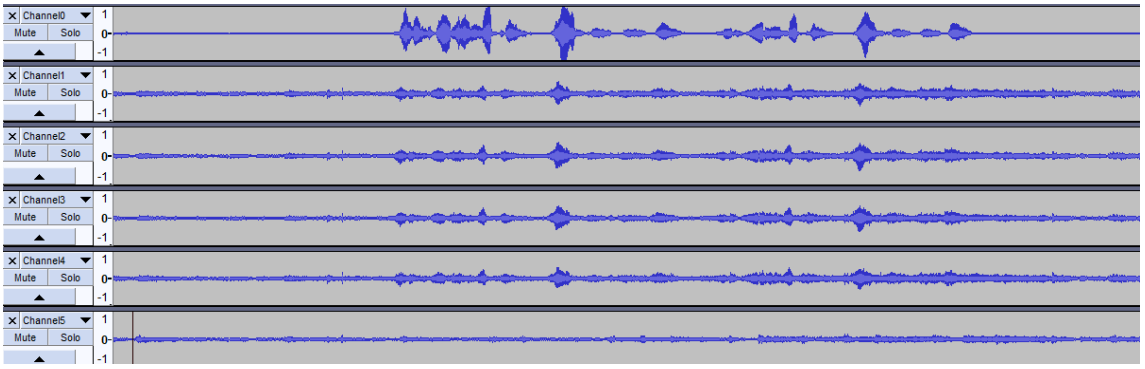
Out of Box Demo

Here is the Acoustic Echo Cancellation example with 6 channels firmware.

- Step 1. Connect the USB cable to PC and audio jack to speaker.

- Step 2. Select the mic array v2.1 as output device in PC side.

- Step 3. Start the audacity to record.
- Step 4. Play music at PC side first and then we talk.
- Step 5. We will see the audacity screen as below, Please click **Solo** to hear each channel audio.



Channel0 Audio(processed by algorithms):

0:00 / 0:12

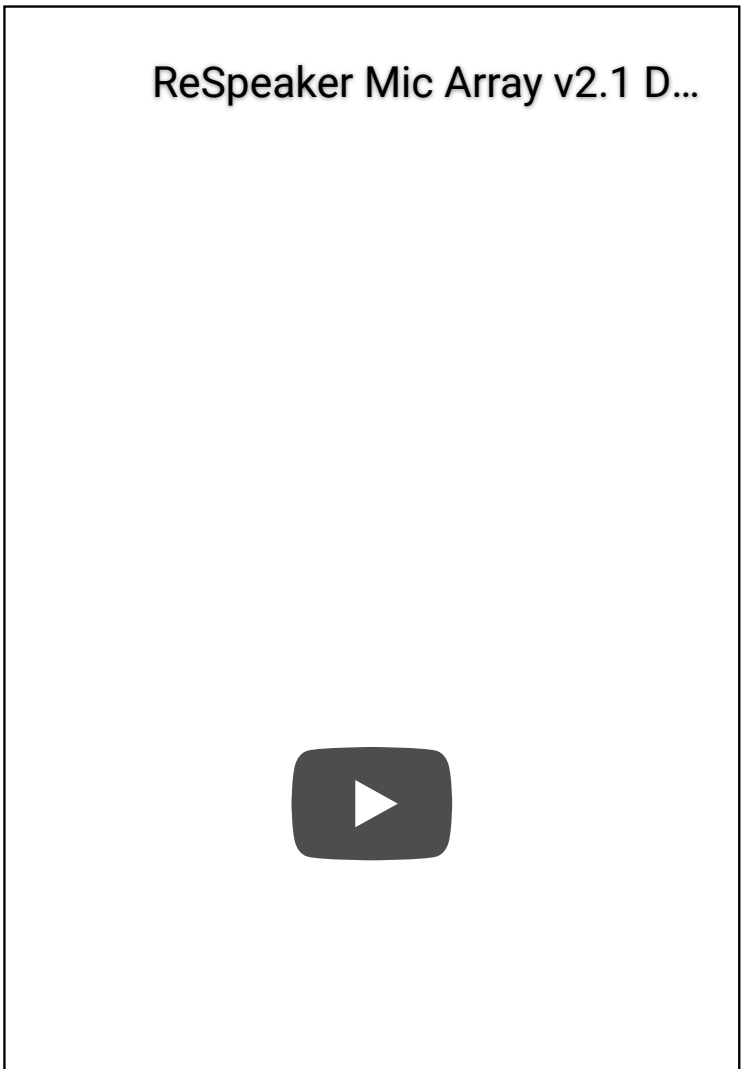
Channel1 Audio(Mic1 raw data):

0:00 / 0:12

Channel5 Audio(Playback data):

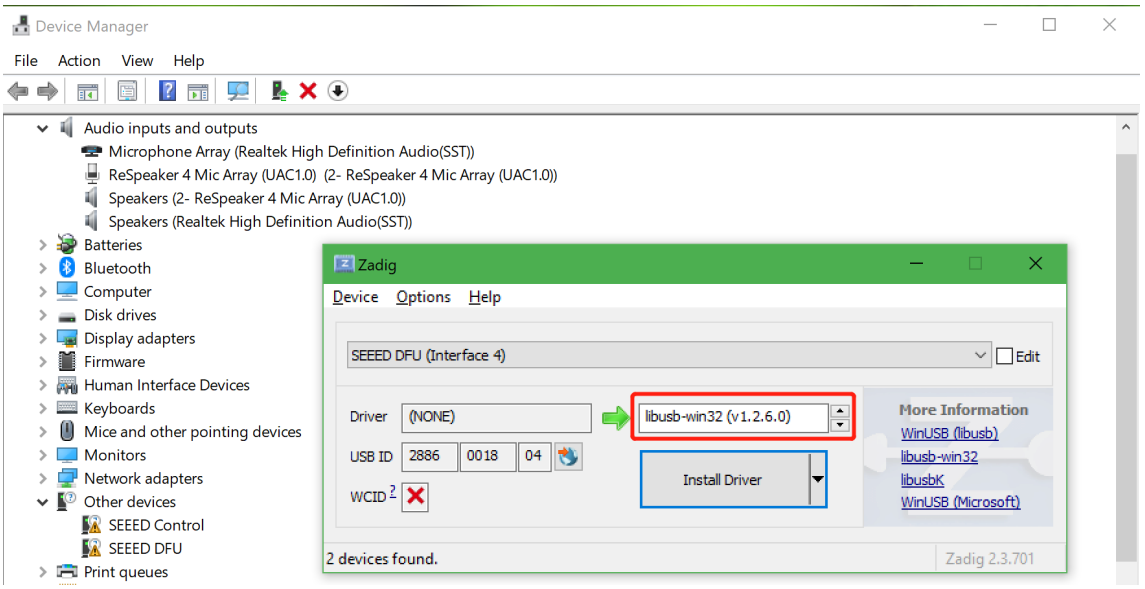
0:00 / 0:12

Here is the video about the DOA and AEC.



Install DFU and LED Control Driver

- **Windows:** Audio recording and playback works well by default. Libusb-win32 driver is only required to control LEDs an DSP parameters on Windows. We use a handy tool - Zadig [http://zadig.akeo.ie/] to install the libusb-win32 driver for both `SEEED DFU` and `SEEED Control` (ReSpeaker Mic Array has 2 devices on Windows Device Manager).



> **Warning**
> Please make sure that libusb-win32 is selected, not WinUSB or libusbK.

- **MAC:** No driver is required.
- **Linux:** No driver is required.

Tuning

For Linux/Mac/Windows: We can configure some parameters of built-in algorithms.

- Get the full list parameters, for more info, please refer to FAQ.

```
git clone https://github.com/respeaker/usb_4_mic_array.gi
cd usb_4_mic_array
python tuning.py -p
```

- Example#1, we can turn off Automatic Gain Control (AGC):

```
sudo python tuning.py AGCONOFF 0
```

- Example#2, We can check the DOA angle.

```
pi@raspberrypi:~/usb_4_mic_array $ sudo python tuning.py
DOAANGLE: 180
```

Control the LEDs

We can control the ReSpeaker USB Mic Array's LEDs through USB. The USB device has a Vendor Specific Class Interface which can be used to send data through USB Control Transfer. We refer pyusb python library [https://github.com/pyusb/pyusb] and come out the usb_pixel_ring python library [https://github.com/respeaker/pixel_ring/blob/master/pixel_ring/usb_pixel_ring_v2.py].

The LED control command is sent by pyusb's usb.core.Device.ctrl_transfer(), its parameters as below：

```
ctrl_transfer(usb.util.CTRL_OUT | usb.util.CTRL_TYPE_VENDOR
```

Here are the usb_pixel_ring APIs.

Command	Data	API	Note
0	[0]	pixel_ring.trace()	trace mode, LE on VAD* and D
1	[red, green, blue, 0]	pixel_ring.mono()	mono mode, se single color, for Red(0xFF0000)
Command	Data	API	Note

			Blue(0x0000FF
2	[0]	pixel_ring.listen()	listen mode, sir but not turn LEI
3	[0]	pixel_ring.speak()	wait mode
4	[0]	pixel_ring.think()	speak mode
5	[0]	pixel_ring.spin()	spin mode
6	[r, g, b, 0] * 12	pixel_ring.customize()	custom mode, own color
0x20	[brightness]	pixel_ring.set_brightness()	set brightness,
0x21	[r1, g1, b1, 0, r2, g2, b2, 0]	pixel_ring.set_color_palette()	set color palett pixel_ring.set_c 0x00ff00) toge pixel_ring.think
0x22	[vad_led]	pixel_ring.set_vad_led()	set center LED: depends on VA
0x23	[volume]	pixel_ring.set_volume()	show volume, r
0x24	[pattern]	pixel_ring.change_pattern()	set pattern, 0 - others - Echo p

For Linux: Here is the example to control the leds. Please follow below commands to run the demo.

```
1  git clone https://github.com/respeaker/pixel_ring.git
```

```
cd pixel_ring
sudo python setup.py install
sudo python examples/usb_mic_array.py
```

Here is the code of the usb_mic_array.py.

```
import time
from pixel_ring import pixel_ring


if __name__ == '__main__':
    while True:

        try:
            pixel_ring.wakeup()
            time.sleep(3)
            pixel_ring.think()
            time.sleep(3)
            pixel_ring.speak()
            time.sleep(6)
            pixel_ring.off()
            time.sleep(3)
        except KeyboardInterrupt:
            break


    pixel_ring.off()
    time.sleep(1)
```

For Windows/Mac: Here is the example to control the leds.

- Step 1. Download pixel_ring.

```
git clone https://github.com/respeaker/pixel_ring.git
cd pixel_ring/pixel_ring
```

- Step 2. Create a led_control.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/led_control.py] with below code and run 'python led_control.py'

```
from usb_pixel_ring_v2 import PixelRing
import usb.core
import usb.util
import time

dev = usb.core.find(idVendor=0x2886, idProduct=0x0018)
print dev
if dev:
    pixel_ring = PixelRing(dev)

    while True:
        try:
            pixel_ring.wakeup(180)
            time.sleep(3)
            pixel_ring.listen()
            time.sleep(3)
            pixel_ring.think()
            time.sleep(3)
            pixel_ring.set_volume(8)
            time.sleep(3)
            pixel_ring.off()
            time.sleep(3)
        except KeyboardInterrupt:
            break

    pixel_ring.off()
```

Note
If you see "None" printed on screen, please reinstall the libusb-win32 driver.

DOA (Direction of Arrival)

For Windows/Mac/Linux: Here is the example to view the DOA. The Green LED is the indicator of the voice direction. For the angle, please refer to hardware overview.

- Step 1. Download the usb_4_mic_array.

```
git clone https://github.com/respeaker/usb_4_mic_array.gi
cd usb_4_mic_array
```

- Step 2. Create a DOA.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/DOA.py] with below code under usb_4_mic_array folder and run 'sudo python DOA.py'

```
from tuning import Tuning
import usb.core
import usb.util
import time

dev = usb.core.find(idVendor=0x2886, idProduct=0x0018)

if dev:
    Mic_tuning = Tuning(dev)
    print Mic_tuning.direction
    while True:
        try:
            print Mic_tuning.direction
            time.sleep(1)
        except KeyboardInterrupt:
            break
```

- Step 3. We will see the DOA as below.

```
pi@raspberrypi:~/usb_4_mic_array $ sudo python doa.py
```

```
184
183
175
105
104
104
103
```

VAD (Voice Activity Detection)

For Windows/Mac/Linux: Here is the example to view the VAD. The Red LED is the indicator of the VAD.

- Step 1. Download the usb_4_mic_array.

```
git clone https://github.com/respeaker/usb_4_mic_array.gi
cd usb_4_mic_array
```

- Step 2. Create a VAD.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/VAD.py] with below code under usb_4_mic_array folder and run 'sudo python VAD.py'

```
from tuning import Tuning
import usb.core
import usb.util
import time

dev = usb.core.find(idVendor=0x2886, idProduct=0x0018)
#print dev
if dev:
    Mic_tuning = Tuning(dev)
    print Mic_tuning.is_voice()
    while True:
```

```
        try:
            print Mic_tuning.is_voice()
            time.sleep(1)
        except KeyboardInterrupt:
            break
```

- Step 3. We will see the DOA as below.

```
pi@raspberrypi:~/usb_4_mic_array $ sudo python VAD.py
0
0
0
1
0
1
0
```

> **Note**
> For the threshold of VAD, we also can use the GAMMAVAD_SR to set. Please refer to Tuning [https://wiki.seeedstudio.com/ReSpeaker_Mic_Array_v2.0/#tuning] for more detail.

Extract Voice

We use PyAudio python library [https://people.csail.mit.edu/hubert/pyaudio/] to extract voice through USB.

For Linux: We can use below commands to record or play the voice.

```
arecord -D plughw:1,0 -f cd test.wav # record, please use
aplay -D plughw:1,0 -f cd test.wav # play, please use the
arecord -D plughw:1,0 -f cd |aplay -D plughw:1,0 -f cd #
```

We also can use python script to extract voice.

- Step 1, We need to run the following script to get the device index number of Mic Array:

```
sudo pip install pyaudio
cd ~
nano get_index.py
```

- Step 2, copy below code and paste on get_index.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/get_index.py].

```
import pyaudio

p = pyaudio.PyAudio()
info = p.get_host_api_info_by_index(0)
numdevices = info.get('deviceCount')

for i in range(0, numdevices):
        if (p.get_device_info_by_host_api_device_index(0,
            print "Input Device id ", i, " - ", p.get_dev
```

- Step 3, press Ctrl + X to exit and press Y to save.
- Step 4, run 'sudo python get_index.py' and we will see the device ID as below.

```
Input Device id  2  -  ReSpeaker 4 Mic Array (UAC1.0): USB A
```

- Step 5, change `RESPEAKER_INDEX = 2` to index number. Run python script record.py

[https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/record.py] to record a speech.

```
import pyaudio
import wave

RESPEAKER_RATE = 16000
RESPEAKER_CHANNELS = 6 # change base on firmwares, 1_chai
RESPEAKER_WIDTH = 2
# run getDeviceInfo.py to get index
RESPEAKER_INDEX = 2  # refer to input device id
CHUNK = 1024
RECORD_SECONDS = 5
WAVE_OUTPUT_FILENAME = "output.wav"

p = pyaudio.PyAudio()

stream = p.open(
            rate=RESPEAKER_RATE,
            format=p.get_format_from_width(RESPEAKER_WID
            channels=RESPEAKER_CHANNELS,
            input=True,
            input_device_index=RESPEAKER_INDEX,)

print("* recording")

frames = []

for i in range(0, int(RESPEAKER_RATE / CHUNK * RECORD_SE
    data = stream.read(CHUNK)
    frames.append(data)

print("* done recording")

stream.stop_stream()
stream.close()
p.terminate()

wf = wave.open(WAVE_OUTPUT_FILENAME, 'wb')
wf.setnchannels(RESPEAKER_CHANNELS)
```

```
wf.setsampwidth(p.get_sample_size(p.get_format_from_widt
wf.setframerate(RESPEAKER_RATE)
wf.writeframes(b''.join(frames))
wf.close()
```

- Step 6. If you want to extract channel 0 data from 6 channels, please follow below code. For other channel X, please change [0::6] to [X::6].

```
import pyaudio
import wave
import numpy as np

RESPEAKER_RATE = 16000
RESPEAKER_CHANNELS = 6 # change base on firmwares, 1_cha
RESPEAKER_WIDTH = 2
# run getDeviceInfo.py to get index
RESPEAKER_INDEX = 3  # refer to input device id
CHUNK = 1024
RECORD_SECONDS = 3
WAVE_OUTPUT_FILENAME = "output.wav"

p = pyaudio.PyAudio()

stream = p.open(
            rate=RESPEAKER_RATE,
            format=p.get_format_from_width(RESPEAKER_WID
            channels=RESPEAKER_CHANNELS,
            input=True,
            input_device_index=RESPEAKER_INDEX,)

print("* recording")

frames = []

for i in range(0, int(RESPEAKER_RATE / CHUNK * RECORD_SE
    data = stream.read(CHUNK)
    # extract channel 0 data from 6 channels, if you wan
```

```
    a = np.fromstring(data,dtype=np.int16)[0::6]
    frames.append(a.tostring())

print("* done recording")

stream.stop_stream()
stream.close()
p.terminate()

wf = wave.open(WAVE_OUTPUT_FILENAME, 'wb')
wf.setnchannels(1)
wf.setsampwidth(p.get_sample_size(p.get_format_from_widt
wf.setframerate(RESPEAKER_RATE)
wf.writeframes(b''.join(frames))
wf.close()
```

For Windows:

- Step 1. We run below command to install pyaudio.

```
pip install pyaudio
```

- Step 2. Use get_index.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/get_index.py] to get device index.

```
C:\Users\XXX\Desktop>python get_index.py
Input Device id  0  -  Microsoft Sound Mapper - Input
Input Device id  1  -  ReSpeaker 4 Mic Array (UAC1.0)
Input Device id  2  -  Internal Microphone (Conexant I)
```

- Step 3. Modify the device index and channels of record.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/record.py] and then extract voice.

```
C:\Users\XXX\Desktop>python record.py
* recording
* done recording
```

> **Warning**
> If we see "Error: %1 is not a valid Win32 application.", please install Python Win32 version.

For MAC:

- Step 1. We run below command to install pyaudio.

```
pip install pyaudio
```

- Step 2. Use get_index.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/get_index.py] to get device index.

```
MacBook-Air:Desktop XXX$ python get_index.py
Input Device id  0  -  Built-in Microphone
Input Device id  2  -  ReSpeaker 4 Mic Array (UAC1.0)
```

- Step 3. Modify the device index and channels of record.py [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/record.py] and then extract voice.

```
MacBook-Air:Desktop XXX$ python record.py
2018-03-24 14:53:02.400 Python[2360:16629] 14:53:02.399 W
* recording
* done recording
```

Realtime Sound Source Localization and Tracking

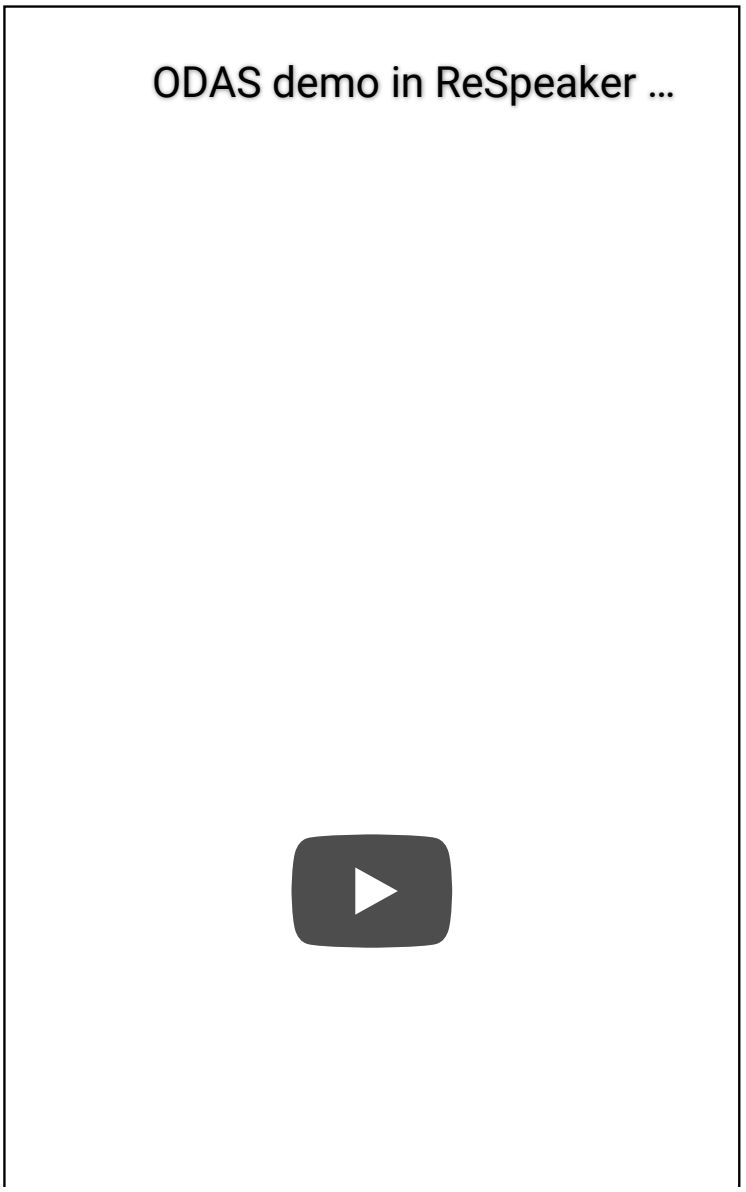
ODAS [https://github.com/introlab/odas] stands for Open embeddeD Audition System. This is a library dedicated to perform sound source localization, tracking, separation and post-filtering. Let's have a fun with it.

For Linux:

- Step 1. Get ODAS and build it.

```
sudo apt-get install libfftw3-dev libconfig-dev libasound
git clone https://github.com/introlab/odas.git
mkdir odas/build
cd odas/build
cmake ..
make
```

- Step 2. Get ODAS Studio [https://github.com/introlab/odas_web/releases] and open it.
- Step 3. The odascore will be at **odas/bin/odaslive**, the **config file** is odas.cfg [https://raw.githubusercontent.com/respeaker/usb_4_mic_array/master/odas.cfg].
- Step 4. Upgrade mic array with 6_channels_firmware.bin which includes 4 channels raw audio data.



FAQ

Q1: Parameters of built-in algorithms

```
pi@raspberrypi:~/usb_4_mic_array $ python tuning.py -p
name            type    max min r/w info
-------------------------------
AECFREEZEONOFF      int 1   0   rw  Adaptive Echo Cancel


AECNORM             float   16  0.25    rw  Limit on nor
```

```
AECPATHCHANGE       int 1   0   ro  AEC Path Change Dete

AECSILENCELEVEL     float   1   1e-09   rw  Threshold fo
AECSILENCEMODE      int 1   0   ro  AEC far-end silence

AGCDESIREDLEVEL     float   0.99    1e-08   rw  Target p

AGCGAIN             float   1000    1   rw  Current AGC

AGCMAXGAIN          float   1000    1   rw  Maximum AGC

AGCONOFF            int 1   0   rw  Automatic Gain Contr

AGCTIME             float   1   0.1 rw  Ramps-up / down
CNIONOFF            int 1   0   rw  Comfort Noise Insert

DOAANGLE            int 359 0   ro  DOA angle. Current v
ECHOONOFF           int 1   0   rw  Echo suppression.

FREEZEONOFF         int 1   0   rw  Adaptive beamformer

FSBPATHCHANGE       int 1   0   ro  FSB Path Change Dete

FSBUPDATED          int 1   0   ro  FSB Update Decision.

GAMMAVAD_SR         float   1000    0   rw  Set the thre

GAMMA_E             float   3   0   rw  Over-subtraction
GAMMA_ENL           float   5   0   rw  Over-subtraction
GAMMA_ETAIL         float   3   0   rw  Over-subtraction
GAMMA_NN            float   3   0   rw  Over-subtraction
GAMMA_NN_SR         float   3   0   rw  Over-subtraction
```

```
GAMMA_NS            float   3   0   rw  Over-subtraction
GAMMA_NS_SR         float   3   0   rw  Over-subtraction

HPFONOFF            int 3   0   rw  High-pass Filter on

MIN_NN              float   1   0   rw  Gain-floor for n

MIN_NN_SR           float   1   0   rw  Gain-floor for n

MIN_NS              float   1   0   rw  Gain-floor for s

MIN_NS_SR           float   1   0   rw  Gain-floor for s

NLAEC_MODE          int 2   0   rw  Non-Linear AEC train

NLATTENONOFF        int 1   0   rw  Non-Linear echo atte

NONSTATNOISEONOFF   int 1   0   rw  Non-stationary noise

NONSTATNOISEONOFF_SR    int 1   0   rw  Non-stationary n

RT60                float   0.9 0.25    ro  Current RT60
RT60ONOFF           int 1   0   rw  RT60 Estimation for
SPEECHDETECTED      int 1   0   ro  Speech detection sta

STATNOISEONOFF      int 1   0   rw  Stationary noise sup

STATNOISEONOFF_SR   int 1   0   rw  Stationary noise sup

TRANSIENTONOFF      int 1   0   rw  Transient echo suppr
```

```


VOICEACTIVITY       int 1   0   ro  VAD voice activity s


```

Q2: ImportError: No module named usb.core

A2: Run sudo pip install pyusb to install the pyusb.

```
pi@raspberrypi:~/usb_4_mic_array $ sudo python tuning.py
Traceback (most recent call last):
  File "tuning.py", line 5, in <module>
    import usb.core
ImportError: No module named usb.core
pi@raspberrypi:~/usb_4_mic_array $ sudo pip install pyusl
Collecting pyusb
  Downloading pyusb-1.0.2.tar.gz (54kB)
    100% |▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒| 61kB 101kB/s
Building wheels for collected packages: pyusb
  Running setup.py bdist_wheel for pyusb ... done
  Stored in directory: /root/.cache/pip/wheels/8b/7f/fe/l
Successfully built pyusb
Installing collected packages: pyusb
Successfully installed pyusb-1.0.2
pi@raspberrypi:~/usb_4_mic_array $ sudo python tuning.py
DOAANGLE: 180
```

Q3: Do you have the example for Raspberry alexa application?

A3: Yes, we can connect the mic array v2.0 to raspberry usb port and follow Raspberry Pi Quick Start Guide with Script [https://github.com/alexa/avs-device-sdk/wiki/Raspberry-Pi-Quick-Start-Guide-with-Script] to do the voice interaction with alexa.

Q4: Do you have the example for Mic array v2.1 with ROS system?

A4: Yes, thanks for Yuki sharing the package for integrating ReSpeaker USB Mic Array with ROS (Robot Operating System) Middleware [https://github.com/furushchev/respeaker_ros].

Q5: How to enable 3.5mm audio port to receive the signal as well as usb port?

A5: Please download the new firmware [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/i2s_i1o2.bin] and burn the XMOS by following How to update firmware [https://wiki.seeedstudio.com/ReSpeaker_Mic_Array_v2.0/#update-firmware].

Q6: #include "portaudio.h" Error when run "sudo pip install pyaudio".

A6: Please run below command to solve the issue.

```
sudo apt-get install portaudio19-dev
```

Resource

- **[PDF]** ReSpeaker USB Mic Array Dimension [https://files.seeedstudio.com/wiki/ReSpeaker-USB-Mics/res/dimension.pdf]
- **[DWG]** ReSpeaker USB Mic Array Case 3D Model [https://files.seeedstudio.com/wiki/ReSpeaker-USB-Mics/res/case.dwg]

- **[PDF]** XVF3000 Product Brief [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/XVF3000-3100-product-brief_1.4.pdf]
- **[PDF]** XVF3000 Datasheet [https://files.seeedstudio.com/wiki/ReSpeaker_Mic_Array_V2/res/XVF3000-3100-TQ128-Datasheet_1.0.pdf]

Tech Support

Please submit any technical issue into our forum [https://forum.seeedstudio.com/].

[https://www.seeedstudio.com/act-4.html?utm_source=wiki&utm_medium=wikibanner&utm_campaign=newproducts]

Farnell AN AVNET COMPANY Newark AN AVNET COMPANY element14

EN - For pricing and availability in your local country please visit one of the below links:

DE - Informationen zu Preisen und Verfügbarkeit in Ihrem Land erhalten Sie über die unten aufgeführten Links:

FR - Pour connaître les tarifs et la disponibilité dans votre pays, cliquez sur l'un des liens suivants:

107990193

EN
This Datasheet is presented by the manufacturer

DE
Dieses Datenblatt wird vom Hersteller bereitgestellt

FR
Cette fiche technique est présentée par le fabricant